Secure API Development and Vulnerability Mitigation Report

Introduction

This project illustrates the development of a simple web API using Flask, the identification of a security vulnerability, and the implementation of a basic security controls to deny unauthorized access. The goal is to understand how insecure design can expose sensitive data and how authentication mechanisms improve security.

Initial Vulnerable API Design

Description

A Flask-based API was created with an endpoint that returned sensitive information stored in a dictionary. The information was login details of various employees of a hypothetical company.

Example Vulnerability

The /data endpoint was publicly accessible without any authentication:

```
@app.route('/data')

def get_data():

    return sensitive_data
```

Security Issue

This design exposed sensitive information to any user who could access the API endpoint.

Risks Identified

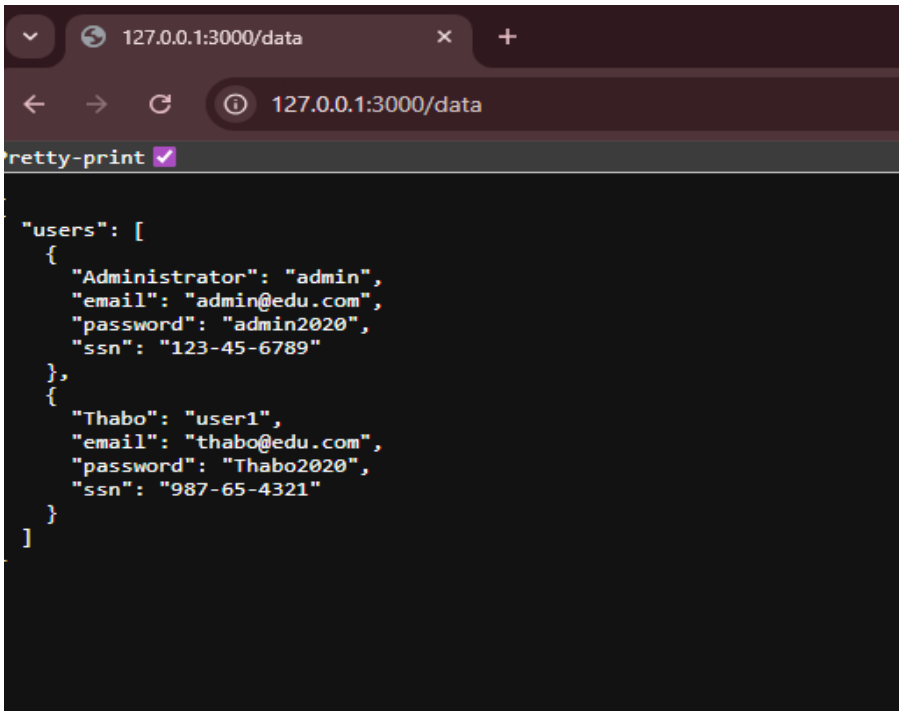
- **Confidentiality breach**: Sensitive data accessible to unauthorized users
- **No access control**: Anyone could retrieve data
- **High exposure risk**: Easily exploitable using a browser or command-line tools

Attack Demonstration

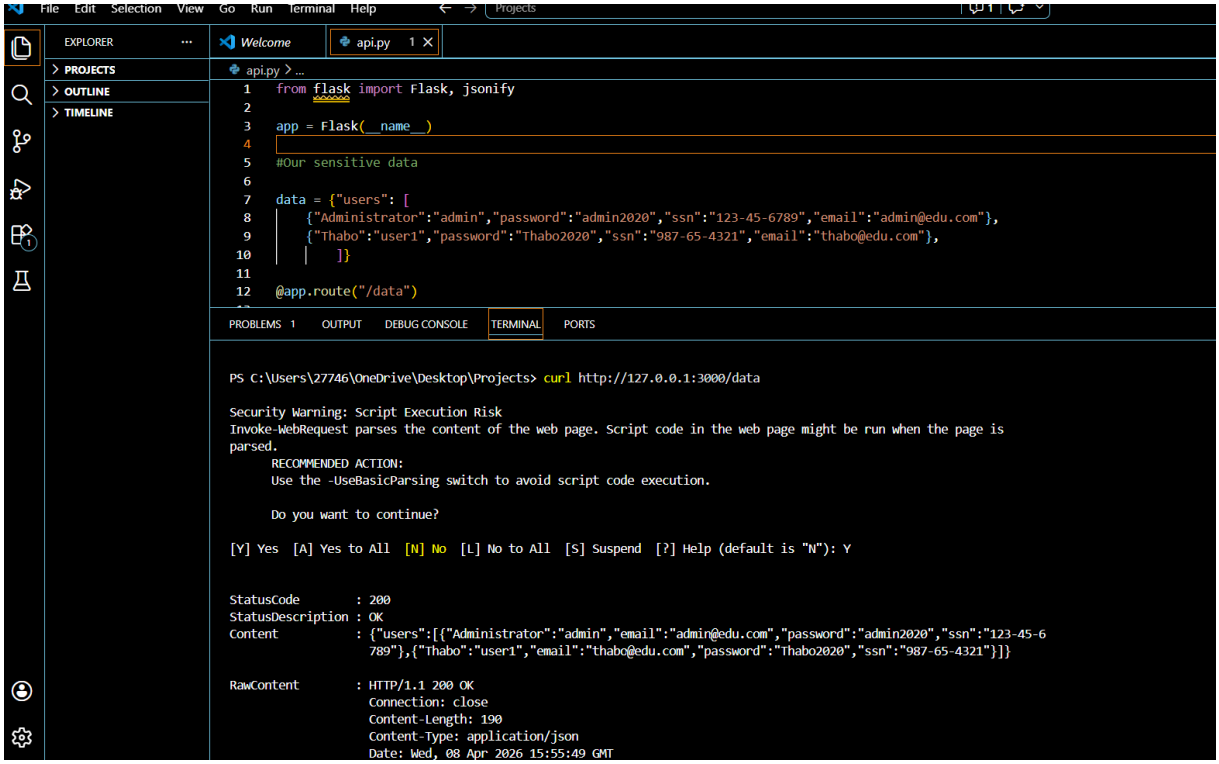
Method

An attacker could retrieve data using:

- Browser access as shown in the picture below indicates ANYONE with access to port 3000 can retrieve ALL sensitive data



- Command-line tools such as PowerShell also shows that curling http://127.0.0.1:3000/data shows one all the information of the employees



Example Attack

curl http://127.0.0.1:3000/data

Outcome

- Full access to sensitive data
- No authentication required

Security Improvement: API Key Authentication

Objective

Restrict access to the API endpoint by requiring a valid API key, this way we can maintain confidentiality by ensuring that only those with the API key can access the employee login detail. This is known as Principle of Least Privilege in security terms.

Implementation Steps

Environment Variable Setup

The API key was stored securely as an environment variable:

```
$env:API_KEY="mySecretKey"
```

Retrieving API Key in Code

```
import os

API_KEY = os.getenv("API_KEY", "mySecretKey")
```

Key Validation Function

```
from flask import request


def is_valid_key():

    return request.headers.get("X-API-Key") == API_KEY
```

Securing the Endpoint

```
from flask import jsonify

@app.route('/data')

def get_data():

    if not is_valid_key():

        return jsonify({"error": "Unauthorized"}), 401


    return jsonify(data)
```

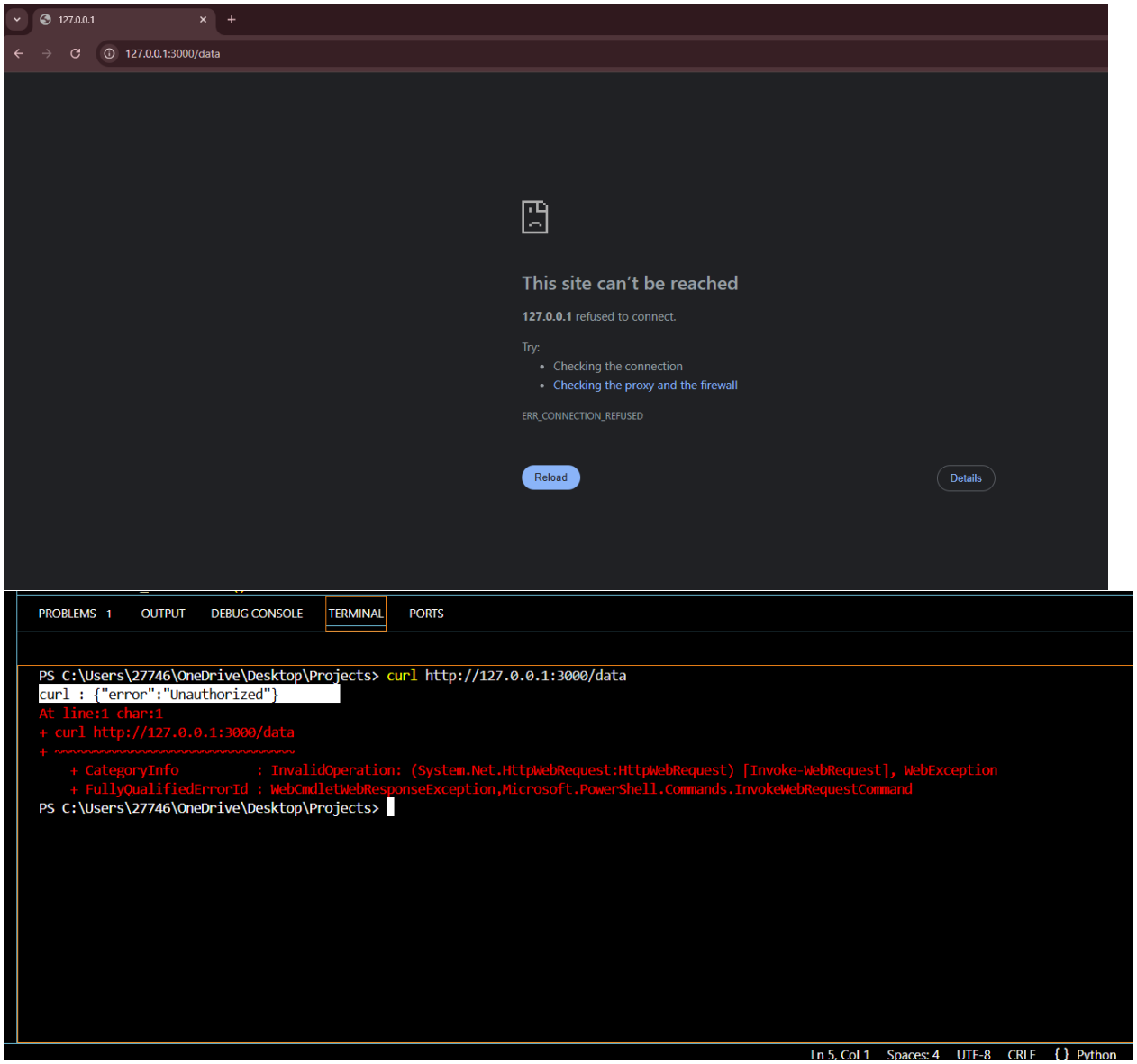
Testing the Secure API

Unauthorized Access Attempt

After securing the API with an API key, we will then attempt to get access the data by accessing port 3000, without using the key since the assumption at this point is that we do not have the API key, we will once again use the command:

curl http://127.0.0.1:3000/data

Result:



{"error": "Unauthorized"} on both the PowerShell and the browser since they both require the API key before giving access to the data.

Authorized Access Attempt

Now, when we attempt to access the data WITH the API Key, as shown in the command below;

(Invoke-WebRequest -Uri http://127.0.0.1:3000/data -Headers @{"X-API-Key"="mySecretKey"}).Content

Result:

```
PS C:\Users\27746\OneDrive\Desktop\Projects> curl -H "X-API-Key: mySecretKey" http://127.0.0.1:3000/data
Invoke-WebRequest : Cannot bind parameter 'Headers'. Cannot convert the "X-API-Key: mySecretKey" value of type "System.String" to type
"System.Collections.IDictionary".
At line:1 char:9
+ curl -H "X-API-Key: mySecretKey" http://127.0.0.1:3000/data
+         ~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument: (:) [Invoke-WebRequest], ParameterBindingException
    + FullyQualifiedErrorId : CannotConvertArgumentNoMessage,Microsoft.PowerShell.Commands.InvokeWebRequestCommand

PS C:\Users\27746\OneDrive\Desktop\Projects> Invoke-WebRequest -Uri http://127.0.0.1:3000/data -Headers @{"X-API-Key"="mySecretKey"}

Security Warning: Script Execution Risk
Invoke-WebRequest parses the content of the web page. Script code in the web page might be run when the page is parsed.
      RECOMMENDED ACTION:
      Use the -UseBasicParsing switch to avoid script code execution.

      Do you want to continue?

[Y] Yes  [A] Yes to All  [N] No  [L] No to All  [S] Suspend  [?] Help (default is "N"): Y


StatusCode        : 200
StatusDescription : OK
Content           : {"users":[{"Administrator":"admin","email":"admin@edu.com","password":"admin2020","ssn":"123-45-6789"},{"Thabo":"user1","email":"thabo@edu.c
                    om","password":"Thabo2020","ssn":"987-65-4321"}]}

RawContent        : HTTP/1.1 200 OK
                    Connection: close
                    Content-Length: 190
                    Content-Type: application/json
                    Date: Wed, 08 Apr 2026 16:59:21 GMT
                    Server: Werkzeug/3.1.8 Python/3.13.3
```

- Access granted
- Sensitive data returned (Highlighted in yellow)

Hashing

The last security measure was to introduce password hashing, whereby plaintext passwords were replaced with hashed values using SHA-256.

Benefits

- Passwords are no longer readable, ensuring an extra level of confidentiality should there be a breach of the API

Security Concepts Demonstrated

Confidentiality

- Initially violated
- Restored through API key protection

Authentication

- Implemented using API key verification

Access Control

- Restricted endpoint access based on valid credentials

Limitations of the Current Solution

- API key is static (not user-specific)
- No encryption (HTTP instead of HTTPS)
- Vulnerable to: Key leakage, Brute force attempts, Header interception

Conclusion

The project successfully demonstrated:

1. How insecure APIs expose sensitive data
2. How attackers can exploit open endpoints
3. That simple authentication mechanisms (API keys) can mitigate risk
4. Hashing can be implemented to ensure an extra layer of data protection

Although basic, this implementation provides a strong foundation for understanding real-world API security and highlights the importance of protecting access to sensitive resources.

Security should be considered from the beginning of system design. Even simple protections like API keys significantly reduce risk but must be combined with stronger mechanisms in real-world systems.